

Compressing Deep Learning Models for Weather Prediction

Ben Trovato*

G.K.M. Tobin*

trovato@corporation.com

webmaster@marysville-ohio.com

Institute for Clarity in Documentation

Dublin, Ohio, USA

Lars Thørväld

The Thørväld Group

Hekla, Iceland

larst@affiliation.org

Valerie Béranger

Inria Paris-Rocquencourt

Rocquencourt, France

Abstract

Deep learning-based weather forecasting models such as FourCastNet have significantly improved the speed and accuracy of numerical weather prediction. However, their large computational and memory requirements make deployment difficult on systems with limited hardware resources. High inference latency, large model size, and heavy GPU memory usage often become major bottlenecks when deploying these models in practical environments.

In this work, we investigate several model compression strategies to improve the efficiency of FourCastNet while maintaining its predictive performance. We explore four techniques: architectural pruning, input channel pruning, FP16 quantization, and a hybrid approach combining pruning and quantization. Architectural pruning reduces model depth by identifying redundant layers using representation similarity between AFNO blocks. Input channel pruning decreases the number of atmospheric variables by performing sensitivity analysis to remove less influential inputs. FP16 quantization reduces the numerical precision of model parameters to lower memory and computation requirements. Experimental results show that these techniques significantly reduce model size and inference latency while preserving forecasting accuracy, making large weather forecasting models more practical for real-world deployment.

CCS Concepts

• **Do Not Use This Code → Generate the Correct Terms for Your Paper;** *Generate the Correct Terms for Your Paper;* Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

Keywords

Model Compression, FourCastNet, Architectural Pruning, Channel Pruning, Quantization, Deep Learning, Weather Forecasting

ACM Reference Format:

Ben Trovato, G.K.M. Tobin, Lars Thørväld, and Valerie Béranger. 2026. Compressing Deep Learning Models for Weather Prediction. In *Proceedings*

*Both authors contributed equally to this research.

Unpublished working draft. Not for distribution.

Permission to make digital or hard copies of all or part of this work for personal or internal use, or the internal or personal use of specific clients, is granted by ACM for libraries and registered users, provided that the fee of \$12.00 is paid directly to ACM. This permission is granted without fee or charge to individuals who are registered with ACM and who have been granted an individual user account by ACM. This permission is granted without fee or charge to individuals who are registered with ACM and who have been granted an individual user account by ACM. This permission is granted without fee or charge to individuals who are registered with ACM and who have been granted an individual user account by ACM.

TAIS, Woodstock, NY

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

<https://doi.org/XXXXXXX.XXXXXXX>

2026-05-16 11:42. Page 1 of 1-6.

of ACM Transactions on AI for Science (TAIS). ACM, New York, NY, USA, 6 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Deep learning has recently transformed many scientific fields, including weather forecasting. Traditional numerical weather prediction models rely on complex physical simulations that require massive computational resources and long processing times. In contrast, data-driven models such as FourCastNet have shown that deep learning can generate weather forecasts much faster while maintaining competitive accuracy. This improvement makes such models promising for real-time forecasting and large-scale climate analysis.

Despite these advantages, deploying large deep learning models in practical environments remains challenging. Models like FourCastNet consist of many layers and millions of parameters, which leads to high inference latency, large memory consumption, and significant storage requirements. These limitations become particularly important when the model needs to run on systems with limited computational resources or when forecasts must be generated repeatedly over long time horizons.

One possible solution to this problem is model compression. Compression techniques aim to reduce the size and computational cost of deep learning models while preserving most of their predictive performance. In this work, we explore multiple compression strategies for improving the efficiency of the FourCastNet model. Specifically, we investigate architectural pruning, input channel pruning, FP16 quantization, and a combination of pruning with quantization.

Our goal is to study how these techniques affect model size, inference latency, and forecasting accuracy. By analyzing these trade-offs, we aim to identify compression strategies that make large weather forecasting models more efficient and practical for real-world deployment.

2 Related Work

As deep learning models continue to grow in size and complexity, model compression has become an important area of research. Large neural networks often achieve high accuracy but require substantial computational resources and memory, which makes their deployment difficult in many practical scenarios [2, 6]. To address this challenge, several techniques have been proposed to reduce model size and computational cost while maintaining performance.

One of the most widely used compression methods is pruning [5]. Pruning removes parameters or structural components of a neural network that contribute little to the final prediction. This can be

done at different levels, such as individual weights, neurons, attention heads, or even entire layers in transformer-based models [9, 12]. Many studies have shown that deep neural networks often contain a significant amount of redundancy, and removing these redundant components can greatly reduce model complexity with minimal impact on accuracy [3].

Quantization [8] is another commonly used technique for model compression. In quantization, the numerical precision of model parameters is reduced, for example from FP32 to lower precision formats such as FP16 or INT8. This helps reduce memory usage and can also improve inference speed, especially on hardware that supports efficient low-precision computation.

Knowledge distillation [7] is another approach that compresses large models by transferring knowledge from a large teacher model to a smaller student model. The smaller model is trained to mimic the predictions of the larger model, allowing it to achieve comparable performance while using fewer parameters.

Recently, deep learning models have also been successfully applied to large-scale weather forecasting. Models such as FourCastNet [10] learn atmospheric dynamics directly from historical weather data and are able to generate forecasts significantly faster than traditional numerical weather prediction systems. However, these models still require large computational resources due to their deep architectures and large number of parameters. This creates a need for efficient compression techniques that can make such models more suitable for deployment.

Motivated by these challenges, our work explores several compression strategies for the FourCastNet model, including architectural pruning, input channel pruning, and quantization. By studying how these techniques affect model efficiency and forecasting performance, we aim to develop a more efficient version of the model while preserving its predictive capability.

3 FourCastNet Overview

Traditional weather forecasting relies on numerical weather prediction (NWP) models that solve complex physical equations, requiring significant computational resources and time [1]. In contrast, deep learning approaches learn atmospheric dynamics directly from historical data, enabling much faster predictions [11]. FourCastNet is a data-driven global weather forecasting model that leverages neural networks to predict atmospheric evolution efficiently while maintaining competitive accuracy [10].

FourCastNet follows a transformer-like architecture designed to process global atmospheric data represented as multi-channel spatial fields. The input is first divided into smaller patches and projected into latent feature representations through a patch embedding layer. These features are then processed by a stack of Adaptive Fourier Neural Operator (AFNO) blocks, which form the core of the model. Finally, a prediction head maps the learned representations back to the spatial domain to generate the forecast for the next time step [10].

The AFNO block is the key component of the model and is responsible for capturing long-range spatial dependencies. Instead of using standard attention mechanisms, AFNO performs token mixing in the frequency domain. The input features are first transformed using a Fast Fourier Transform (FFT), followed by learnable

Table 1: Atmospheric variables used as input channels in the FourCastNet model.

Category	Variables	Description
Surface	u10, v10	10 m wind components
	t2m	2 m air temperature
	sp, mslp	Surface pressure variables
Upper-Air Dynamics	t850	Temperature at 850 hPa
	u1000, v1000	Wind at 1000 hPa
	z1000	Geopotential height at 1000 hPa
	u850, v850	Wind at 850 hPa
	z850	Geopotential height at 850 hPa
	u500, v500	Wind at 500 hPa
	z500	Geopotential height at 500 hPa
	t500	Temperature at 500 hPa
Moisture	z50	Geopotential height at 50 hPa
	r500, r850	Relative humidity at 500 and 850 hPa
	tcwv	Total column water vapor

Table 2: Parameter distribution across major components of the FourCastNet model.

Model Component	Parameters	Percentage
AFNO Backbone (12 Blocks)	60,281,856	80.7%
Parameters per AFNO Block	≈ 5,023,488	–
Embedding + Head + Positional Encoding	14,409,984	19.3%
Total Parameters	74,691,840	100%

filtering in the frequency space, and then transformed back to the spatial domain. This approach allows the model to efficiently capture global atmospheric patterns while maintaining computational efficiency [4].

The model takes global atmospheric data on a latitude–longitude grid, where multiple variables such as wind, temperature, pressure, humidity, and geopotential height are stacked as input channels. The complete set of input variables is summarized in Table 1. These inputs are divided into patches, allowing the model to efficiently process high-resolution global data while preserving spatial structure.

FourCastNet generates forecasts by predicting the next atmospheric state given the current input. By recursively feeding predictions back as inputs, the model can produce multi-step forecasts in an autoregressive manner [10].

Despite its efficiency compared to traditional NWP models, FourCastNet remains computationally intensive. A large portion of the model complexity is concentrated in the AFNO backbone, which accounts for approximately 80.7% of the total parameters, with each block containing around 5 million parameters. The parameter distribution is summarized in Table 2.

Although FourCastNet provides fast and accurate forecasts, its large size and computational cost pose challenges for deployment. Since most of the parameters are concentrated in the AFNO backbone, reducing redundancy in this component is a natural direction for improving efficiency. In this work, we explore architectural

pruning, input channel pruning, and quantization to address this challenge.

4 Experimental Setup

We conduct our experiments using the ERA5 dataset, which provides global atmospheric reanalysis data across multiple variables and pressure levels. The dataset is widely used for training and evaluating data-driven weather forecasting models. In our experiments, we use data from 2014–2015 for fine-tuning and 2018 for evaluation.

We evaluate model performance using the root mean squared error (RMSE) and anomaly correlation coefficient (ACC) of geopotential height at 500 hPa (Z500). This metric is commonly used to assess large-scale atmospheric prediction accuracy.

All models are evaluated in a single-step forecasting setting, where the model predicts the next time step based on the current atmospheric state. For pruned models, we perform fine-tuning after pruning to recover performance. The evaluation is conducted consistently across all configurations to ensure fair comparison.

All experiments are implemented in PyTorch and conducted on an NVIDIA A100 GPU with 80GB memory. We evaluate models in both FP32 and FP16 precision settings to analyze the impact of quantization on performance.

Inference latency is measured per prediction step with a batch size of 1 under consistent conditions across all models. All experiments are conducted in the same environment to ensure fair comparison of latency and performance across different compression methods.

5 Compression Methods

5.1 Architectural Pruning

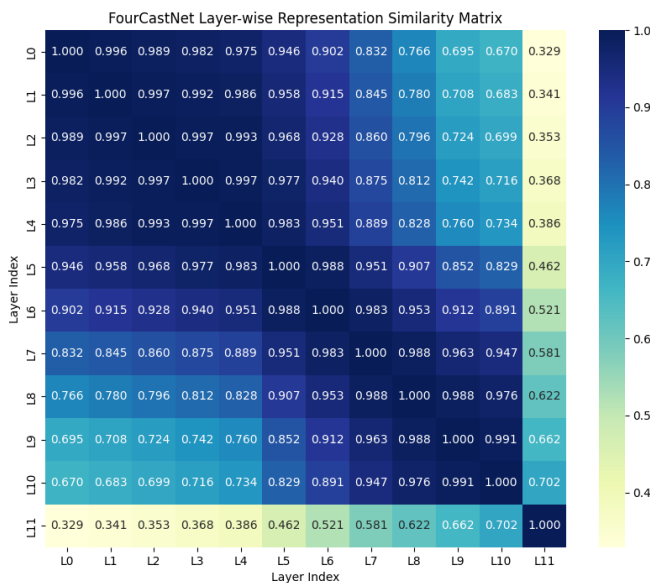


Figure 1: Similarity matrix heatmap

The computational cost of FourCastNet is dominated by the AFNO backbone, which accounts for more than 80% of the model parameters and most of the inference latency. This makes the transformer blocks a natural target for compression. Reducing the number of layers can significantly improve efficiency, but it is important to remove only redundant layers to avoid performance degradation.

To identify redundant layers, we use a similarity-based pruning approach. For each AFNO block, we extract intermediate feature representations and compute pairwise cosine similarity. This results in a similarity matrix that captures how similar different layers are. Layers with cosine similarity greater than 0.99 are considered highly redundant, as they are likely learning overlapping information. Figure 1 shows the similarity matrix, where several layers exhibit similar behavior.

Based on this analysis, layers are ranked according to their similarity, and the most redundant ones are removed. In our experiments, the pruning order is [3, 4, 2, 1, 10, 9]. Starting from the baseline model with 12 AFNO blocks, we prune multiple layers at once by selecting the top-k most similar layers. This approach allows controlled evaluation across different compression levels. We evaluate configurations by removing up to 6 layers, with the 6-layer model (50% depth reduction) providing a good balance between efficiency and performance.

After pruning, we fine-tune only the remaining AFNO blocks using the same ERA5 dataset for up to 6 epochs. Fine-tuning is performed using the Adam optimizer with a learning rate scheduler. This design reduces training cost while adapting the core representations of the model. We observe that most of the performance recovery occurs within the first 1–2 epochs, with diminishing improvements in later epochs. Each epoch of fine-tuning takes approximately 30 minutes.

Pruning leads to significant improvements in efficiency. The baseline model has a latency of 142 ms and 74.7M parameters, while the 6-layer model reduces latency to 73 ms (1.95x speedup) and parameters to 44.5M (40% reduction). However, pruning increases Z500 RMSE as more layers are removed. Fine-tuning helps recover much of this loss, especially for moderate pruning levels, as shown in Table 3.

Overall, these results confirm that the AFNO backbone contains substantial redundancy. Moderate pruning provides strong efficiency gains with limited impact on accuracy, while aggressive pruning leads to noticeable degradation.

5.2 Input Channel Pruning

The FourCastNet model takes 20 atmospheric variables as input, each represented as a separate channel. However, not all variables contribute equally to the model’s predictions, and some may be less important. Reducing the number of input channels can simplify the input representation and reduce computation in the early stages of the model.

To identify less important variables, we perform a leave-one-out sensitivity analysis. In this approach, each variable is removed individually, and the impact on forecasting performance is measured using Z500 RMSE. Based on this analysis, variables are ranked according to their importance, as shown in Table 4.

Table 3: Z500 RMSE (m^2/s^2) across different pruning levels before and after fine-tuning

# Pruned	Base	Fine-Tuning Epochs (2014–2015)						Latency (ms)
		1	2	3	4	5	6	
0	36.97	34.68	34.57	34.49	35.58	34.69	34.95	142 (1.0x)
1	191.10	37.0	36.53	36.72	35.64	36.22	35.00	130 (1.0x)
2	365.04	41.49	39.71	38.79	38.78	37.98	37.44	119 (1.09x)
3	569.02	48.54	44.21	42.80	41.88	41.0	40.63	107 (1.19x)
4	817.76	64.61	53.87	48.74	46.88	45.06	44.27	96 (1.32x)
5	1061.37	70.49	58.74	54.33	50.57	48.76	47.17	84 (1.48x)
6	1297.22	84.56	69.99	60.07	56.63	54.68	52.51	73 (1.95x)

Table 4: Sensitivity analysis of input variables based on Z500 RMSE.

Category	Variable	RMSE	ACC
I. Non-Essential (Lowest Sensitivity)			
	r500	39.21	0.9993
	r850	44.00	0.9991
	t500	47.02	0.9989
	v10	53.46	0.9987
	v1000	56.28	0.9985
II. Low Impact			
	v850	68.38	0.9981
	u1000	101.33	0.9952
	u850	101.96	0.9952
	u10	122.38	0.9925
	sp	123.32	0.9935
	z50	124.38	0.9940
	u500	134.02	0.9928
III. Moderate Impact			
	tcwv	215.99	0.9785
	mslp	328.39	0.9484
IV. Critical			
	t2m	742.57	0.7583
	t850	794.60	0.7691
	z1000	2330.40	0.2560
V. Mandatory (Highest Sensitivity)			
	z850	5662.06	0.1853
	z500	8980.41	-0.1836
	v500	10375.14	-0.0402

The analysis shows that some variables have very low impact on performance and can be considered redundant, while others are critical and cannot be removed without significant degradation. In particular, moisture-related variables show the least sensitivity, whereas key dynamical variables such as z500 and v500 are the most important.

Based on these findings, we remove low-impact variables and reduce the number of input channels from 20 to 14, while retaining the most important variables. This leads to approximately 30% reduction in the parameters of the patch embedding layer. However, since the majority of parameters are concentrated in the AFNO backbone, the overall reduction in model size is only about 0.4%, and there is no significant improvement in inference latency.

Overall, input channel pruning helps in understanding feature importance and simplifying the input space, but its impact on overall efficiency is limited. It serves as a complementary approach to architectural pruning.

5.3 Quantization (FP32 to FP16)

The baseline FourCastNet model uses 32-bit floating point (FP32) precision, which results in high memory usage and large storage requirements. This can be a limitation when deploying the model in resource-constrained environments. Reducing the numerical precision of model parameters provides a simple way to compress the model without modifying its architecture.

We apply quantization by converting model weights from FP32 to half precision (FP16). This reduces the number of bits used to represent each parameter, leading to lower memory usage and faster computation on supported hardware. For quantization to be effective, the entire computation pipeline, including input tensors, must be executed in FP16.

Quantization leads to significant improvements in both storage and inference efficiency. The model size is reduced from approximately 284.93 MB to 140 MB, resulting in nearly a 50% reduction. In terms of latency, the inference time decreases from 142.18 ms to 46.72 ms per step, providing nearly a 3x speedup.

Importantly, the impact on forecasting performance is minimal. The Z500 RMSE changes only slightly from 37.07 to 37.28, and the Z500 ACC remains almost unchanged (0.9994 to 0.9993), indicating that the predictive capability of the model is preserved.

The results show that quantization is highly effective for improving efficiency without affecting accuracy. However, the latency gains are only realized when the full computation is performed in FP16. If the model is partially executed in FP32, the performance improvement is not observed.

Overall, quantization provides a substantial reduction in model size and inference latency, making it one of the most effective compression techniques in this work. It complements architectural pruning and can be combined with it for further efficiency improvements.

5.4 Combined Architectural Pruning and Quantization

Architectural pruning and quantization improve different aspects of model efficiency. Pruning reduces the number of layers and parameters, while quantization reduces memory usage and improves

Table 5: Performance comparison of pruned models under FP32 and FP16 precision.

# Pruned	Fine-Tuned FP32		Fine-Tuned FP16	
	Z500 RMSE	Latency (ms)	Z500 RMSE	Latency (ms)
1	35.00	130	35.33	42.98
2	37.44	119	37.70	39.19
3	40.63	107	40.71	35.44
4	44.27	96	44.40	31.57
5	47.17	84	47.27	27.86
6	52.51	73	52.46	24.07

computation speed. Combining both methods allows us to achieve higher overall efficiency by addressing both structural and numerical redundancy in the model.

We first apply architectural pruning to remove redundant AFNO blocks based on similarity analysis. The pruned models are then converted from FP32 to FP16. For each pruning level, we evaluate the model in both FP32 and FP16 to understand the combined effect of pruning and quantization on performance.

Table 5 presents the performance of pruned models under both FP32 and FP16 settings. The results show that quantization consistently reduces latency across all pruning levels. For example, in the case of 6 pruned layers, the latency decreases from 73 ms in FP32 to 24.07 ms in FP16, providing nearly a 3× speedup.

At the same time, the impact on accuracy is minimal. The Z500 RMSE remains almost unchanged across FP32 and FP16 configurations for all pruning levels. This indicates that quantization does not significantly affect forecasting performance, even when combined with aggressive pruning.

The results show that combining pruning and quantization provides the maximum improvement in efficiency. While pruning reduces model size and baseline latency, quantization further accelerates inference without introducing noticeable accuracy loss.

Across all pruning levels, FP16 consistently achieves lower latency compared to FP32, while maintaining similar RMSE values. This demonstrates that quantization complements pruning effectively.

Overall, the combined approach achieves the best trade-off between efficiency and accuracy. The results suggest that applying quantization on top of a pruned model is an effective strategy for deployment. In particular, moderately pruned models combined with FP16 offer significant latency reduction while maintaining competitive forecasting performance.

6 Results and Discussion

We evaluate different compression techniques in terms of inference latency and model size. Architectural pruning reduces latency from 142 ms to 73 ms (2× speedup) by removing redundant AFNO blocks. Quantization further improves efficiency, reducing latency to 46.7 ms (3× speedup) and halving model size. The combined approach achieves the best performance, with latency as low as 24 ms. In contrast, input channel pruning has negligible impact on latency.

Architectural pruning increases Z500 RMSE as more layers are removed, but fine-tuning recovers much of the performance, especially for moderate pruning. Quantization has minimal impact on

accuracy, with RMSE remaining nearly unchanged. The combined approach provides a good balance, where moderate pruning with FP16 maintains competitive accuracy while significantly reducing latency.

Among the methods, architectural pruning effectively reduces model size and latency, while quantization provides the largest speedup with negligible accuracy loss. Input channel pruning has limited effect on efficiency but helps identify less important variables. The combined method delivers the best overall performance.

The AFNO backbone contains significant redundancy, enabling effective pruning. Quantization is highly efficient with minimal accuracy impact, and combining both methods yields the highest efficiency gains.

Moderate pruning combined with FP16 quantization provides the best trade-off between efficiency and accuracy, making the model more suitable for real-world deployment.

7 Conclusion

In this work, we explored different model compression techniques to improve the efficiency of the FourCastNet weather forecasting model. While FourCastNet provides fast and accurate predictions, its large size and computational requirements make deployment challenging.

We investigated architectural pruning, input channel pruning, and FP16 quantization, along with a combined approach. Our results show that the AFNO backbone contains significant redundancy, allowing effective depth reduction with manageable impact on accuracy. Quantization proved to be highly effective, providing substantial improvements in latency and memory usage with almost no loss in performance. In contrast, input channel pruning had limited impact on overall efficiency but helped identify less important variables.

Among all methods, combining pruning with quantization achieved the best results, offering the highest reduction in latency while maintaining competitive accuracy. Overall, our findings demonstrate that carefully designed compression techniques can make large-scale weather forecasting models more efficient and practical for real-world deployment.

In future work, more advanced compression methods such as lower-precision quantization and structured pruning can be explored to further improve efficiency.

Acknowledgments

To Robert, for the bagels and explaining CMYK and color spaces.

References

- [1] Peter Bauer, Alan Thorpe, and Gilbert Brunet. 2015. The quiet revolution of numerical weather prediction. *Nature* (2015).
- [2] Yu Cheng et al. 2017. A Survey of Model Compression and Acceleration for Deep Neural Networks. *IEEE Signal Processing Magazine* (2017).
- [3] Jonathan Frankle and Michael Carbin. 2019. The Lottery Ticket Hypothesis. *ICLR* (2019).
- [4] John Guibas et al. 2021. Efficient Token Mixing for Transformers via Fourier Transforms. *NeurIPS* (2021).
- [5] Song Han et al. 2015. Learning both Weights and Connections for Efficient Neural Networks. *NeurIPS* (2015).
- [6] Song Han et al. 2016. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. *ICLR* (2016).
- [7] Geoffrey Hinton et al. 2015. Distilling the Knowledge in a Neural Network. *NeurIPS Workshop* (2015).
- [8] Benoit Jacob et al. 2018. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. *CVPR* (2018).
- [9] Paul Michel et al. 2019. Are Sixteen Heads Really Better than One? *NeurIPS* (2019).
- [10] Jaideep Pathak et al. 2022. FourCastNet: A Global Data-driven High-resolution Weather Model. *arXiv preprint arXiv:2202.11214* (2022).
- [11] Markus Reichstein et al. 2019. Deep learning and process understanding for data-driven Earth system science. *Nature* (2019).
- [12] Elena Voita et al. 2019. Analyzing Multi-Head Self-Attention. *ACL* (2019).

A Research Methods

A.1 Part One

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Morbi malesuada, quam in pulvinar varius, metus nunc fermentum urna,

id sollicitudin purus odio sit amet enim. Aliquam ullamcorper eu ipsum vel mollis. Curabitur quis dictum nisl. Phasellus vel semper risus, et lacinia dolor. Integer ultricies commodo sem nec semper.

A.2 Part Two

Etiam commodo feugiat nisl pulvinar pellentesque. Etiam auctor sodales ligula, non varius nibh pulvinar semper. Suspendisse nec lectus non ipsum convallis congue hendrerit vitae sapien. Donec at laoreet eros. Vivamus non purus placerat, scelerisque diam eu, cursus ante. Etiam aliquam tortor auctor efficitur mattis.

B Online Resources

Nam id fermentum dui. Suspendisse sagittis tortor a nulla mollis, in pulvinar ex pretium. Sed interdum orci quis metus euismod, et sagittis enim maximus. Vestibulum gravida massa ut felis suscipit congue. Quisque mattis elit a risus ultrices commodo venenatis eget dui. Etiam sagittis eleifend elementum.

Nam interdum magna at lectus dignissim, ac dignissim lorem rhoncus. Maecenas eu arcu ac neque placerat aliquam. Nunc pulvinar massa et mattis lacinia.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009